CODE :

(Banker’s Algorithm) :-

```
#include <stdio.h>

int main() {
    int n, m;
    int allocation[20][20], max[20][20], need[20][20];
    int available[20], work[20], finish[20] = {0};
    int safeSequence[20];
    int i, j, count = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("\nEnter Available Resources:\n");
    for (j = 0; j < m; j++) {
        scanf("%d", &available[j]);
        work[j] = available[j];
    }

    // Calculate Need Matrix
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Display Need Matrix
    printf("\nNeed Matrix:\n");
    for (i = 0; i < n; i++) {
```

```
    for (j = 0; j < m; j++) {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}

// Banker's Algorithm
while (count < n) {
    int found = 0;

    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int canExecute = 1;

            // Check if Need[i] <= Work
            for (j = 0; j < m; j++) {
                if (need[i][j] > work[j]) {
                    canExecute = 0;
                    break;
                }
            }

            if (canExecute) {
                // Process can finish
                for (j = 0; j < m; j++) {
                    work[j] += allocation[i][j];
                }

                safeSequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    // No process can proceed
    if (found == 0) {
        break;
    }
}

// Check whether system is safe
if (count == n) {
    printf("\nSystem is in a SAFE state.\n");

    printf("Safe Sequence: ");
    for (i = 0; i < n; i++) {
        printf("P%d", safeSequence[i]);

        if (i != n - 1)
            printf(" -> ");
    }
```

```
        printf("\n");
    } else {
        printf("\nSystem is in an UNSAFE state.\n");
        printf("Deadlock may occur.\n");
    }

    return 0;
}
```

OUTPUT :

